

JAVA Concurrency Utilities (from Jakob Jenkov's webpage: jenkov.com)

BlockingQueue

- represents a queue which is thread safe to put elements into, and take elements out of from. In other words, multiple threads can be inserting and taking elements concurrently from a Java BlockingQueue, without any concurrency issues arising.
- The term blocking queue comes from the fact that the Java BlockingQueue is capable of blocking the threads that try to insert or take elements from the queue.
- Since BlockingQueue is an interface, you need to use one of its implementations to use it.
 - ArrayBlockingQueue
 - a bounded, blocking queue that stores the elements internally in an array.
 - DelayQueue
 - blocks the elements internally until a certain delay has expired.
 - LinkedBlockingQueue
 - keeps the elements internally in a linked structure (linked nodes).
 - PriorityBlockingQueue
 - SynchronousQueue
 - a queue that can be used to exchange a single element with another thread. A thread inserting an element into the queue is blocked until another thread takes that element from the queue.

ConcurrentMap

- represents a Map which is capable of handling concurrent access (puts and gets) to it.
- ConcurrentHashMap
 - Since ConcurrentMap is an interface, you need to use one of its implementations (ConcurrentHashMap)
 - The ConcurrentHashMap is very similar to the java.util.HashMap class, except that ConcurrentHashMap offers better concurrency than HashMap does.

CountDownLatch

- allows one or more threads to wait for a given set of
- A CountDownLatch is initialized with a given count. This count is decremented by calls to the countDown() method. Threads waiting for this count to reach zero can call one of the await() methods. Calling await() blocks the thread until the count reaches zero.

CyclicBarrier

- a barrier that all threads must wait at, until all threads reach it, before any of the threads can continue.

Exchanger

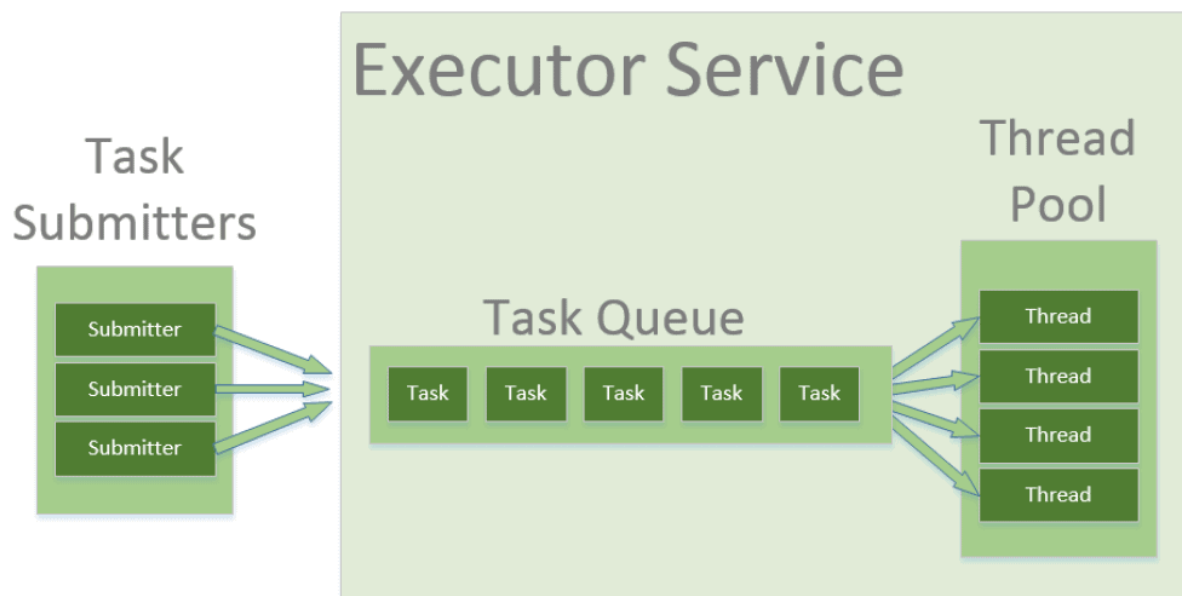
- represents a kind of rendezvous point where two threads can exchange objects.

Semaphore

- The counting semaphore is initialized with a given number of "permits". For each call to `acquire()` a permit is taken by the calling thread. For each call to `release()` a permit is returned to the semaphore. Thus, at most N threads can pass the `acquire()` method without any `release()` calls, where N is the number of permits the semaphore was initialized with. The permits are just a simple counter.
- counting semaphore / binary semaphore

Thread Pool (Concept)

- a pool of threads that can be reused to execute tasks, so that each thread may execute more than one task
- comparison: "creating a new thread per task" vs. "thread pool reusing existing threads for task execution"
 - Creating a new thread comes with a performance overhead compared to reusing a thread that is already created.
 - That is why reusing an existing thread to execute a task can result in a higher total throughput than creating a new thread per task.
 - Thread pool makes it easy to control how many threads are active at a time.
- Built-in thread pool: `ExecutorService`
- We can submit parallel tasks for execution to a instance of a thread pool



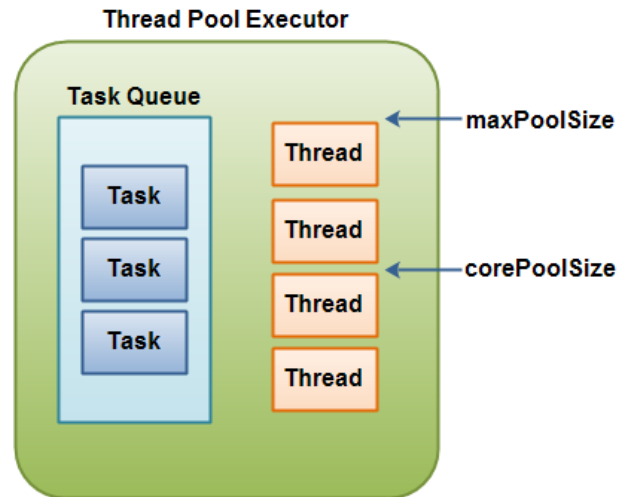
ExecutorService

- Java `ExecutorService` interface (ex code: `ArraySumWithExecutor.java` in our class webpage.)

- represents an asynchronous execution mechanism which is capable of executing tasks concurrently in the background.
- Once the thread has delegated the task to the `ExecutorService`, the thread continues its own execution independent of the execution of that task.
- `execute()` / `submit()`
 - Both functions can be used for submit tasks for execution
 - `execute()`: just execute. you don't care about the result.
 - `submit()`: you can get the result (return value) of the task. You can check task completion through **Future**. (see the sample code: `ArraySumWithExecutor.java`)
 - `submit()` takes a task (either `Runnable` or `Callable`), pushes it into a thread pool's queue, and immediately returns a `Future` object
 - Non-blocking: the thread that calls `submit()` does not wait for the task to finish; it keeps running its own code.
- **Executors**: a helper class that can be used to create preconfigured thread pool instances.
- **ThreadPoolExecutor**
 - `ThreadPoolExecutor` is an implementation of the `ExecutorService` interface.
 - The thread pool contained inside the `ThreadPoolExecutor` can contain a varying amount of threads. The number of threads in the pool is determined by these variables: `corePoolSize`, `maximumPoolSize`

```
int corePoolSize = 5;
int maxPoolSize = 10;
long keepAliveTime = 5000;

ExecutorService threadPoolExecutor =
    new ThreadPoolExecutor(
        corePoolSize,
        maxPoolSize,
        keepAliveTime,
        TimeUnit.MILLISECONDS,
        new LinkedBlockingQueue<Runnable>()
    );
```



Java Future

- Represents the result of an asynchronous computation.
- When the asynchronous task is created, a Java **Future** object is returned.
- Once the asynchronous task completes, the result can be accessed via the **Future** object returned when the task was started.
- Important methods of Future
 - `get()`: it retrieves the result. If the task is not done yet, the calling thread will block (pause) here until the result is ready.
 - `isDone()`: returns true if the task completed, failed, or was cancelled.
 - `cancel()`: attempts to stop the task

Java Callable

- Represents an asynchronous task which can be executed by a separate thread.
- For instance, it is possible to submit a **Callable** object to a **Java ExecutorService** which will then execute it asynchronously.
- `call()` method is called in order to execute the asynchronous task. The `call()` method can return a result
- **Callable vs Runnable**
 - **Callable** is similar to **Runnable** interface, in both of them represents a task that is intended to be executed concurrently by a separate thread
 - **Runnable's** `run()` method does not return a value.
 - **Runnable** was originally designed for long running concurrent execution, while **Callable** is more designed for one-off tasks that return a single result.

Java ForkJoinPool

- **Fork-Join Model:** based on **divide-and-conquer**
 - **fork:** break a task into smaller subtasks
 - **join:** solve each subtask and combine the results

- **ForkJoinPool** schedules and runs these subtasks in parallel using worker threads.
- It uses a **work-stealing algorithm**, where idle threads "steal" tasks from busy threads to improve efficiency.
- Often more efficient than manually creating threads for recursive tasks.
- Code example: `ArraySum.java`

Lock

- Interface (you cannot create an instance of Lock directly.). You must create an instance of a class that implements the Lock interface (`ReentrantLock`).
- represents a concurrent lock which is used to guard against race conditions inside critical sections.

```
-----
Lock lock = new ReentrantLock();
lock.lock();
    //critical section
lock.unlock();
-----
```

- First a Lock is created. Then its `lock()` method is called. Now the Lock instance is locked. Any other thread calling `lock()` will be blocked until the thread that locked the lock calls `unlock()`. Finally `unlock()` is called, and the Lock is now unlocked so other threads can lock it. Obviously all threads must share the same Lock instance. If each thread creates its own Lock instance, then they will be locking on different locks, and thus not be blocking each other from access.
- Fail-safe Lock and Unlock
 - To avoid exceptions locking forever, you should lock and unlock within try-finally block
- Lock Reentrance
 - A lock is called reentrant if the thread that holds the lock can lock it again. A non-reentrant lock is a lock which cannot be locked again if locked
- Lock Fairness and Starvation
 - An unfair lock does not guarantee the order of threads access for lock. That means that a waiting thread could risk waiting forever, if other threads keep trying to lock the lock, and are given priority over the waiting thread. This situation can lead to starvation.
 - In a fair lock, threads acquire the lock in the order they request it.
 - Lock starvation occurs when a particular thread attempts to acquire a lock and never succeeds because another thread is already holding the lock.
 - `ReentrantLock` behaviour is unfair by default. However, you can tell it to operate in fair mode via its constructor.
- `tryLock()`
 - The `tryLock()` method attempts to lock the Lock instance immediately. It returns true if the locking succeeds, false if Lock is already locked. This method never blocks.

ReadWriteLock

- an advanced thread lock mechanism that allows multiple threads to read a certain resource, but only one to write it, at a time.
- A read-write lock allows for a greater level of concurrency in accessing shared data than that permitted by a mutual exclusion lock.
- The idea is that multiple threads can read from a shared resource without causing concurrency errors. The concurrency errors first occur when reads and writes to a shared resource occur concurrently, or if multiple writes take place concurrently.
 - Read Lock (Shared Lock): allowed to lock if no threads have locked the ReadWriteLock for writing, and no thread has requested a write lock (but not yet obtained it). Thus, multiple threads can lock the lock for reading.
 - Write Lock (Exclusive Lock): allowed to lock if no threads are reading or writing. Thus, only one thread at a time can lock the lock for writing.
- Sample code

```
-----  
ReadWriteLock readWriteLock = new ReentrantReadWriteLock();  
  
readWriteLock.readLock().lock();  
    // multiple readers can enter this section  
    // if not locked for writing, and not writers waiting  
    // to lock for writing.  
readWriteLock.readLock().unlock();  
  
readWriteLock.writeLock().lock();  
    // only one writer can enter this section,  
    // and only if no threads are currently reading.  
readWriteLock.writeLock().unlock();  
-----
```

AtomicInteger

- provides you with a int variable which can be read and written atomically
- also contains advanced atomic operations like compareAndSet()
 - This method compares the current value of the AtomicInteger instance to an expected value, and if the two values are equal, sets a new value for the AtomicInteger instance.
- Supports various atomic operations
 - addAndGet()
 - getAndAdd()
 - getAndIncrement()
 - incrementAndGet()
 - decrementAndGet()
 - getAndDecrement()

AtomicReference

- provides an object reference variable which can be read and written atomically.
- particularly useful when managing shared objects accessed and modified by multiple threads.

AtomicIntegerArray

- represents an array of int . The int elements in the AtomicIntegerArray can be updated atomically.
- Supports various atomic operations for the elements of the array
 - get()
 - set()
 - compareAndSet()
 - addAndGet() , getAndAdd()
 - incrementAndGet() , getAndIncrement()
 - decrementAndGet() , getAndDecrement()